МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования
**«Южно-Уральский государственный университет
(национальный исследовательский университет)»**
Высшая школа электроники и компьютерных наук
Кафедра системного программирования

ОТЧЕТ
по производственной практике
(научно-исследовательская работа)

Выполнил,
студент группы КЭ-403
_____ Е.В. Елисеев

Научный руководитель,
профессор кафедры СП,
д.ф.-м.н., доцент
_____ М.Л. Цымблер

Руководитель практики:
доц.каф.СП, к.ф.-м.н.__
Турлакова С.У._____
Дата: _____
Оценка: _____

Челябинск, 2024 г.

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Южно-Уральский государственный университет
Кафедра системного программирования

УТВЕРЖДАЮ
Зав. кафедрой СП
_____ Л.Б. Соколинский

ЗАДАНИЕ
на производственную практику
(научно-исследовательскую работу)

1. Тема работы

Разработка компонентов и интерфейса системы поиска шаблонов в данных портала GitHub.

2. Исходные данные к работе

3.1. Han J., Kamber M., Pei J. Data Mining: Concepts and Techniques - Morgan Kaufmann Publishers, 2012. – 703 p.
3.2. McKinney W. Python for Data Analysis - O'Reilly Media, 2018. – 540 p.
3.3. GitHub [Электронный ресурс]. URL: https://github.com (дата обращения: 02.02.2024 г.).

3. Перечень подлежащих разработке вопросов

3.1. Провести анализ и спецификацию предметной области.
3.2. Выполнить проектирование архитектуры компонентов и пользовательского интерфейса системы.
3.3. Реализовать компоненты системы.

4. Сроки

Дата выдачи задания: 1 февраля 2024 г.
Срок сдачи законченной работы: 22 февраля 2024 г.

Руководитель практики со стороны ЮУрГУ:
Доцент кафедры СП, к.ф.-м.н. Турлакова С.У.

Научный руководитель практики:
Профессор кафедры СП д.ф.-м.н., доцент Цымблер М.Л.

Задание принял к исполнению: Елисеев Е.В.

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ 4
1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ 6
1.1.Описание предметной области 6
1.2.Обзор работ по тематике 8
2. ПРОЕКТИРОВАНИЕ СИСТЕМЫ 11
2.1.Требования к системе 11
2.2.Разработка пользовательского интерфейса 12
2.3.Архитектура системы 16
3. РЕАЛИЗАЦИЯ СИСТЕМЫ 18
3.1.Программные средства реализации 18
3.2.Реализация компонентов системы 18
ЗАКЛЮЧЕНИЕ 19
СПИСОК ЛИТЕРАТУРЫ 20
ПРИЛОЖЕНИЕ 21

ВВЕДЕНИЕ

Актуальность работы

В современном мире информационных технологий, данные играют ключевую роль в принятии решений, исследованиях и разработке продуктов. Портал GitHub [1] являющийся крупнейшим в мире хранилищем кода, предоставляет огромное количество данных на различных ресурсах, которые могут быть использованы для анализа как самими сотрудниками сервиса, так и сторонними пользователями. Поиск зависимостей в этих данных может помочь в понимании тенденций разработки, популярности определенных технологий, поведения разработчиков и многого другого.

Было создано огромное число методов, позволяющих извлекать зависимости из данных. Поиск шаблонов [2] является важным методом по нахождению зависимостей и закономерностей в данных, и применяется в большом числе сфер от маркетинга до генетики. Применение метода поиска шаблонов на данных GitHub также может помочь в получении ценной, актуальной информации. Например, можно постараться выявить, какого типа репозитории становятся популярными, а какого нет, какие факторы положительно влияют на популярность, а какие наоборот чаще всего оказывают отрицательное влияние и делают репозиторий неинтересным для других пользователей.

Создание приложения – рабочего инструмента для поиска шаблонов в данных GitHub, с помощью которого можно было бы получать и анализировать самые свежие данные, могло бы помочь руководителям проектов или другим пользователям лучше понимать текущее состояние сферы разработки, применять важные бизнес-решения в проектах, опираясь на актуальную информацию, а также понять, как поднять популярность репозитория, привлечь разработчиков или спонсоров в свой проект.

Постановка задачи

Целью практики является продвижение в задаче разработки системы поиска шаблонов в данных портала GitHub. Для достижения поставленной цели необходимо решить следующие задачи:

1. Провести анализ и спецификацию предметной области.
2. Выполнить проектирование архитектуры компонентов и пользовательского интерфейса системы.
3. Реализовать компоненты системы.

1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1. Описание предметной области

GitHub – это облачная платформа для хранения репозиториев, основанная на системе контроля версий Git [3]. Обычно репозиторий GitHub может включать в себя исходный код программы, документацию, графические ресурсы и другие элементы проекта. Репозиторий также отслеживает все изменения, которые вносятся в файлы, и кто их внес, что облегчает совместную работу и управление версиями. Каждый репозиторий имеет свой уникальный URL, который можно использовать для доступа к нему через веб-интерфейс GitHub или через Git на локальной машине.

Описание исходных данных

Когда пользователь выполняет какое-либо действие с репозиторием в GitHub, это действие сохраняется и связывается с конкретным событием. GitHub, а также сторонние сервисы, предоставляют доступ к событиям и другим данным, если они касаются публичного репозитория. Ниже приведены возможные способы получения данных.

GH Archive [4] – это сервис по сбору и хранению данных публичных репозиториев, который также предоставляет доступ к этим данным по часам. Однако, получение данных на этом портале не предполагает возможности составления детализированного запроса и позволяет загрузить только данные всех событий, которые произошли за час, что усложняет отбор на более широкий промежуток времени. Также данные GH Archive доступны в более удобном варианте, публичном датасете Google BigQuery [5], который позволяет производить SQL запросы, что освобождает от скачивания лишних данных. Однако BigQuery имеет довольно строгие ограничения для запросов, которые не позволяют сканировать и запрашивать большой объем данных бесплатно.

Одним из лучших вариантов запроса большого объема данных с небольшим числом ограничений является сервис ClickHouse Playground [6],

с помощью которого можно получить доступ к уже предобработанным данным из GH Archive путем отправки SQL-запросов.

Например, можно получить такие данные, как:

- имя репозитория;
- количество сделанных commit;
- количество сделанных push;
- средний объем push;
- количество звезд репозитория;
- число присоединившихся участников и т.д.;

Преобразовав эти данные, можно получить товары, которые будут составлять будущий набор транзакций. Также можно усложнить задачу и путем преобразования вычислить более сложные товары, отражающие, например, соотношение слитых pull request ко всем pull request, или соотношение закрытых issue ко всем issue. В итоге, можно получить следующий пример транзакции для каждого репозитория, который указан в таблице 1.

Таблица 1 – Пример возможной транзакции

Items				
Высокое число commit	Среднее число push	Много участников	Много закрытых issue	Много звезд

Применив алгоритм, по поиску ассоциативных правил в таком наборе транзакций, можно, к примеру, получить шаблоны, представленные в таблице 2.

Таблица 2 – Пример возможных шаблонов

Антецедент	**Консеквент**
Много участников, Много закрытых issue	Много звезд
Высокое число push, Много незакрытых issue	Мало звезд

Приведенные выше шаблоны, например, могут обнаружить возможные зависимости между активностью репозитория и количеством

звезд, что возможно позволит определить, какого типа активность команды влияет на рост популярности репозитория.

Рассмотрим еще один вариант получения данных GitHub. GitHub API [7] – это официальный набор способов и правил, по которым различные программы общаются с GitHub и обмениваются данными. С его помощью также можно выполнять HTTP-запросы с целью получения различных данных в формате JSON о каком-либо репозитории, пользователе или событии, если они носят публичный характер.

Ниже представлены примеры, данных, которые можно получить:

- name – название репозитория;
- description – описание репозитория;
- is_fork – является ли репозиторий fork;
- language – язык, использующийся в репозитории;
- license_name – имя лицензии репозитория.

Некоторые данные из представленных могут дополнить транзакцию, такими товарами, как «Язык программирования Python» или «Лицензия MIT». При поиске шаблонов это может помочь определить, какие языки или виды лицензий лучше влияют на популярность репозитория, что может помочь разработчикам или менеджерам проектов делать правильный выбор при создании своего проекта.

1.2. Обзор работ по тематике

Анализу данных портала Github посвящено много работ. В основном при анализе используются различные алгоритмы машинного обучения, такие, как KNN или нейронные сети. Ниже приведен обзор некоторых работ.

Целью работы [8] является анализ данных GitHub для моделирования развивающихся репозиториев и динамических трендов в программировании. Авторы используют данные сервиса GH Archive о 9350 популярных репозиториев с 2015 по 2018 год, включая количество часто

встречающихся событий, таких, как fork, pull request, push и т.д., язык программирования. Анализ проводится с помощью метода анализа временных рядов на основе нейронной сети LSTM [9] (Long Short Term Memory), которая специально предназначена для анализа нелинейных временных рядов.

Проблема анализа данных GitHub и прогнозирования трендов разбивается на три подзадачи: прогнозирование трендов репозиториев, прогнозирование трендов языков программирования и прогнозирование трендов областей применения. В каждой из этих подзадач используются различные данные для построения временных рядов.

Производительность экспериментов измеряется в терминах среднеквадратической ошибки RMSE и коэффициента детерминации R2. Модели показали хорошую производительность с очень низким значением RMSE и высоким значением R2, которое лежит между 0 и 1.

По результатам анализа было замечено, что репозитории в областях веб-майнинга, машинного обучения и информационной безопасности становятся все более популярными. Из языков программирования JavaScript является наиболее трендовым языком.

Важно отметить, что LSTM была выбрана для этой задачи из-за ее эффективности в решении последовательных и временных проблем без проблемы исчезающего градиента, которая влияет на производительность других моделей, таких как RNN (Recurrent Neural Network).

В работе [10] изучается влияние пандемии COVID-19 на работу удаленных команд. Основные задачи включают изучение общего эффекта пандемии, оценку эффектов пандемии на отдельные команды и анализ свойств команд, которые связаны с устойчивостью под воздействием шока.

Авторы использовали данные о 71928 репозитории для референсного набора и 87914 репозиториев для целевого из долгосрочного журнала событий GHArchive и зеркала официального источника GHTorrent [11]. На основе данных о количестве push событий репозитория, вычисляется

метрика, отражающая продуктивность команды, а на основе количества участников вычисляется рост команды.

Были обучены модели машинного обучения градиентного бустинга деревьев решений GBDT и случайного леса для прогнозирования результатов за определенный месяц в 2019 году, а затем были сделаны контрфактические прогнозы результатов целевых репозиториев за шесть месяцев 2020 года. Эффект пандемии затем измерялся по разнице между наблюдаемыми и контрфактическими результатами.

В исследовании было обнаружено, что продуктивность команды сразу снизилась после шока (1-3 месяцы), а затем восстановилась к норме через несколько месяцев (4-6 месяцы). Количество активных участников команды, с другой стороны, не сразу упало, когда наступила пандемия (1-3 месяцы), но оно снижается постепенно, когда команды реагируют в более долгосрочной перспективе (5-6 месяцы).

Из приведенных выше работ, можно заметить, что основным и самым популярным источником данных является GH Archive. Авторы работ собирают данные по каждому репозиторию и за внушительный срок - от полу года до нескольких лет, что объяснятся тем, что данные, из которых можно сделать какой-либо вывод, накапливаются за достаточно большой срок. Эти особенности стоит учесть при будущей разработке системы.

2. ПРОЕКТИРОВАНИЕ СИСТЕМЫ

2.1. Требования к системе

В ходе проектирования были определены функциональные и нефункциональные требования к разрабатываемой системе.

Функциональные требования

Функциональные требования устанавливают требования о том, как должна себя вести система и прямым образом относятся к функционалу системы.

Разрабатываемая система должна удовлетворять следующим функциональным требованиям.

1. Система должна предоставлять возможность запроса и загрузки выбранных данных из API GitHub в базу данных.
2. Система должна предоставлять возможность выбора параметров отбора запрашиваемых из API GitHub данных.
3. Система должна предоставлять возможность работы для двух пользовательских ролей – администратора и обычного пользователя.
4. Система должна предоставлять возможность запроса ранее загруженных данных в программу из базы данных.
5. Система должна предоставлять возможность выбора содержания транзакции для поиска шаблонов.
6. Система должна предоставлять возможность крайних значений метрик для поиска шаблонов.
7. Система должна предоставлять возможность просмотра и фильтрации шаблонов.
8. Система должна предоставлять возможность сохранения шаблонов.
9. Система должна предоставлять удобный пользовательский интерфейс.

Нефункциональные требования

Нефункциональные требования устанавливают требования ограничений для системы, которые не влияют прямым образом на функционал системы.

Система должна удовлетворять следующим нефункциональным требованиям.

1. Система должна представлять собой веб-приложение.
2. Система должна быть написана на языке Python.

2.2. Разработка пользовательского интерфейса

Варианты использования

Для описания способов взаимодействия с системой была разработана диаграмма вариантов использования на языке UML, представленная на рисунке 1.

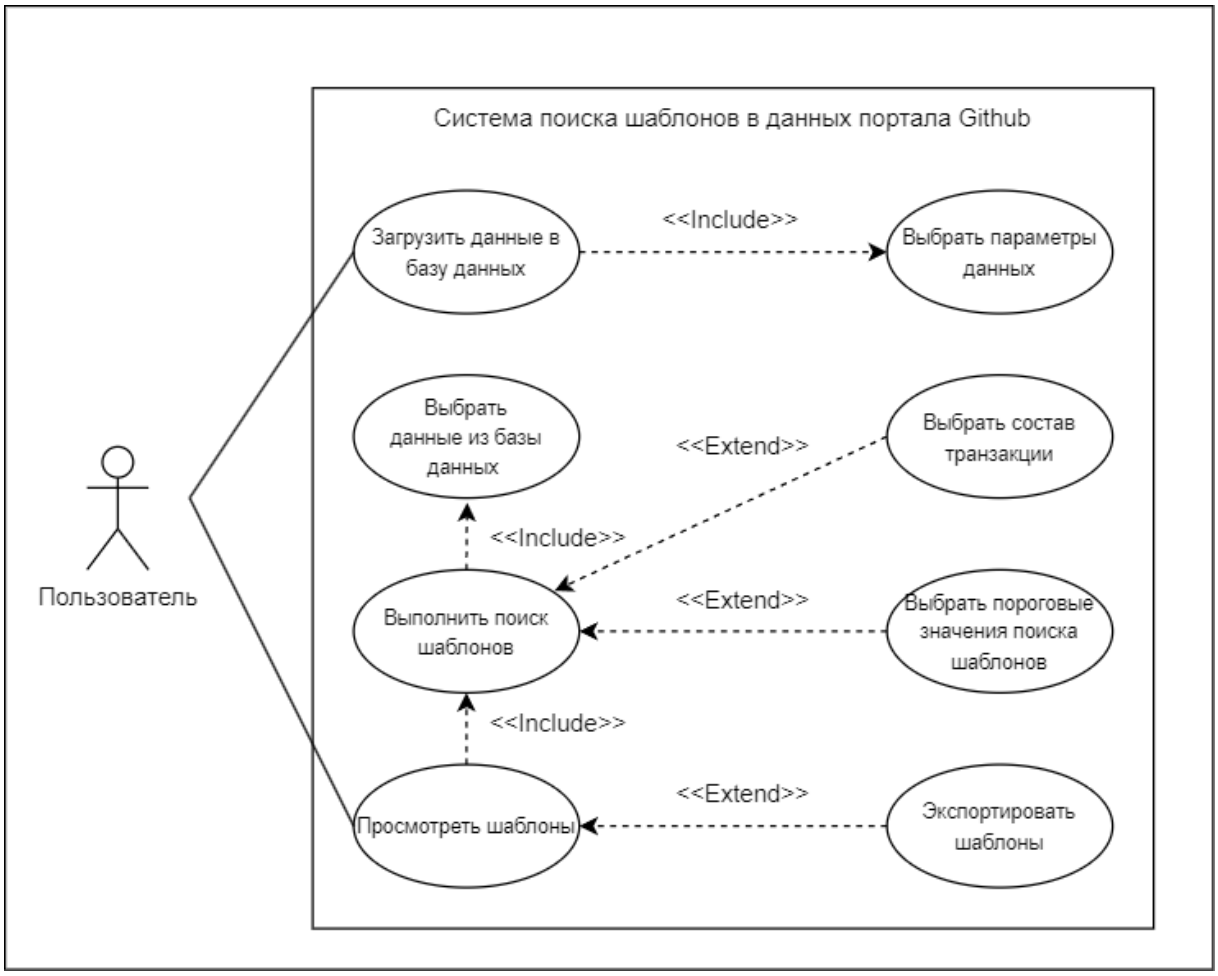

Рисунок 1 – Диаграмма вариантов использования

Единственным актером системы является обычный пользователь, который может использовать все функции приложения. Ниже приведены эти функции:

1. Выбрать параметры запрашиваемых данных. Пользователь может выбрать параметр запрашиваемых из API данных.
2. Загрузить данные в базу данных. Пользователь может выполнить запрос к сервисам, которые хранят данные и сохранить результаты запроса в базу данных.
3. Выбрать данные из базы данных. Пользователь выбирает данные из базы данных для поиска шаблонов.
4. Выбрать состав транзакции. Пользователь выбирает товары, из которых будет состоять будущая транзакция.
5. Выбрать пороговые значения поиска шаблонов. Пользователь устанавливает пороговые значения для поиска шаблонов (поддержка, достоверность, подъем).
6. Выполнить поиск шаблонов. Пользователь выполнят поиск шаблонов с заданными параметрами.
7. Просмотреть шаблоны. Пользователь может просмотреть и проанализировать найденные шаблоны.
8. Экспортировать шаблоны. Пользователь может экспортировать шаблоны в нужном формате.

Макет интерфейса

Интерфейс приложения будет представлять собой одну большую страницу и выполнен в Figma [12]. Макет первой части страницы изображен на рисунке 2.

На первой части страницы расположен интерфейс для выбора параметров данных. В самом начале можно авторизоваться через GitHub, это нужно, чтобы программа могла запрашивать некоторые данные без ограничений. Пользователь может выбрать промежуток времени, за который ему нужны данные, а также параметры репозиториев.

Рисунок 2 – Макет первой части страницы

У каждой опции есть клавиша подсказки, которая может отобразить дополнительную информацию. Далее пользователь выбирает состав транзакции, товары, которые он хочет включить, может изменить их названия и квантили. Последним этапом пользователь может сохранить выбранные данные, тогда отобразиться следующий раздел страницы, либо же выбрать загруженные ранее данные в программу. Макета меню загрузки данных изображено на рисунке 3.

В меню будут отображаться ранее сохраненные данные, а также данные, которые пользователь только что сохранил. Выделив, выбранные строки, пользователь может нажать кнопку «Загрузить». Также пользователь может выбрать несколько наборов данных, тем самым, объединив их.

Рисунок 3 – Макет меню загрузки данных

На рисунке 4 расположен макет второй части интерфейса страницы.

Рисунок 4 – Макет второй части интерфейса страницы

Эта часть открывается после сохранения пользователем данных или после выбора загруженных ранее данных. Здесь пользователь может

выбрать количество элементов слева (в антецеденте) и справа (консеквенте), а также указать минимальные значения параметров поддержки, достоверности и подъема. После вышеперечисленного, пользователь может нажать кнопку «Искать шаблоны», после которой программа выполнит поиск и отобразится таблица с шаблонами. В таблице доступен фильтр данных в колонке «Антецедент» и «Консеквент» по элементам, а также сортировка по убыванию и возрастанию в колонках с метриками. Также пользователь может нажать кнопку «Скачать шаблоны», после чего скачается файл в формате xlsx.

2.3. Архитектура системы

Система будет представлять собой веб-приложение, состоящее из нескольких модулей: головной модуль, модуль интерфейса, модуль запроса данных, модуль работы с базой данных, модуль преобразования данных, модуль поиска шаблонов.

На рисунке 4 изображена диаграмма компонентов системы.

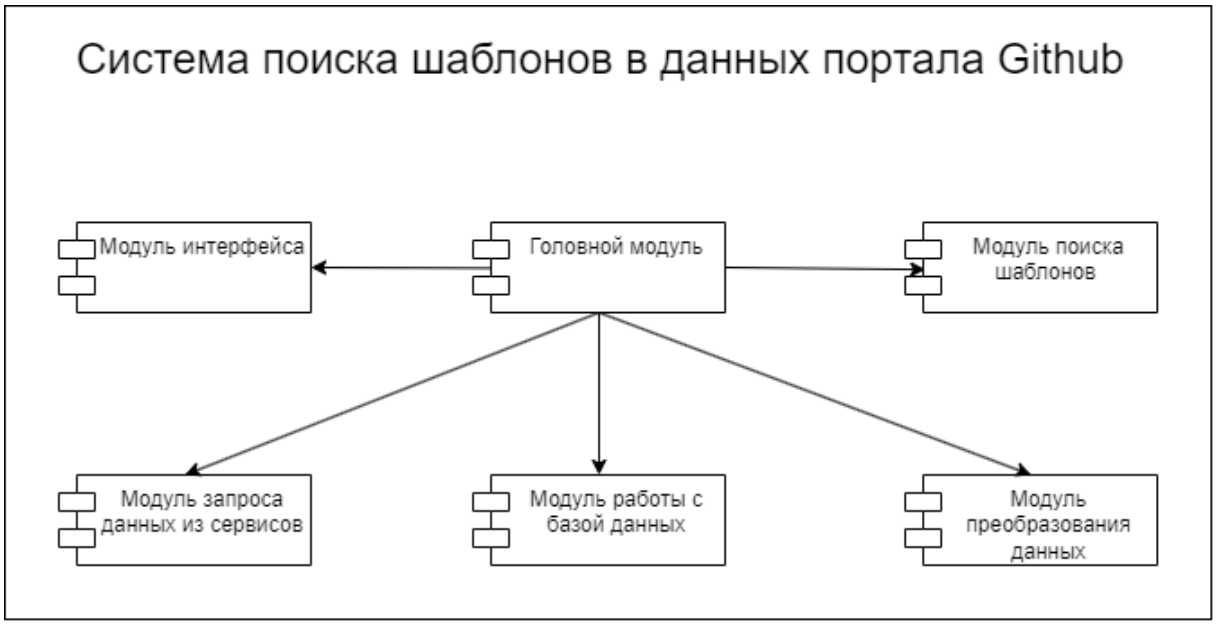

Рисунок 4 – Диаграмма компонентов системы

Головной модуль будет объединять все остальные модули системы.

Модуль интерфейса будет отображать окна веб-приложения, с которыми взаимодействует пользователь.

Модуль запроса данных из сервисов будет взаимодействовать с сервисом Clickhouse Playground, а также с GitHub API. Сначала будет установлено, данные какого типа репозиториев будут запрашиваться, а также промежуток времени. Например это могут быть только репозитории, созданные за этот промежуток, репозитории, где больше одного участника или все вообще все репозитории. Далее по каждому репозиторию будут запрашиваться данные, которые будут составлять будущую транзакцию. Также модуль будет запрашивать данные квантилей для некоторых числовых атрибутов, для будущей трансформации данных в транзакции.

Модуль работы с базой данных будет взаимодействовать с базой данных. У пользователя будет возможность сохранить запрошенные данные в базу данных, чтобы не потерять их и выполнить анализ позже.

Модуль преобразования данных будет содержать функционал для обработки запрошенных данных, удалять дубликаты, пропущенные и ненужные для поиска шаблонов данные, а также преобразовывать данные в вид транзакций.

Модуль поиска шаблонов будет осуществлять поиск шаблонов в транзакциях по заданным пороговым значениям поддержки, достоверности и подъема. Также пользователь сможет выбирать число элементов в антецеденте и консеквенте.

3. РЕАЛИЗАЦИЯ СИСТЕМЫ

3.1. Программные средства реализации

Для разработки программной части был выбран язык Python версии 3.11.7, разработка велась в редакторе кода Visual Studio Code.

3.2. Реализация компонентов системы

За время практики были реализованы следующие компоненты системы:

- модуль запроса данных из сервисов;
- модуль преобразования данных
- модуль поиска шаблонов

Интерфейсы некоторых методов реализованных компонентов находятся в приложении.

ЗАКЛЮЧЕНИЕ

В рамках текущей практики были выполнены следующие задачи:

1. проведен анализ и спецификация предметной области;
2. выполнено проектирование архитектуры компонентов и пользовательского интерфейса;
3. выполнена реализация части компонентов.

Результаты текущей практики заложили основу в разработке системы поиска шаблонов в данных GitHub, что составит значительную часть от выпускной квалификационной работы.

СПИСОК ЛИТЕРАТУРЫ

1. GitHub [Электронный ресурс] URL: https://github.com/ (дата обращения: 02.02.2024 г.).
2. Agrawal R., Imielinski T., Swami A. Mining Association Rules between Sets of Items in Large Databases // ACM SIGMOD Record, 1993. –V. 22. – 207-216 pp.
3. Git documentation [Электронный ресурс]. URL: https://git-scm.com/doc (дата обращения: 02.02.2024 г.).
4. GitHub Archive [Электронный ресурс]. URL: https://www.gharchive.org/ (дата обращения: 02.02.2024 г.).
5. Google BigQuery [Электронный ресурс]. URL: https://cloud.google.com/bigquery (дата обращения: 02.02.2024 г.).
6. ClickHouse Playground [Электронный ресурс]. URL: https://play.clickhouse.com/play?user=play (дата обращения: 02.02.2024 г.).
7. GitHub API documentation [Электронный ресурс]. URL: https://docs.github.com/ru/rest?apiVersion=2022-11-28 (дата обращения: 02.02.2024 г.).
8. Varuna T., Anuraj M. Trend Prediction of GitHub using Time Series Analysis // 10th ICCCNT, 2019.
9. Schmidhuber J., Hochreiter S. Long Short-Term Memory // Neural Computation, 1997. –V. 9. – 1735-1780 pp.
10. Xuan L., Wei A., Yixin W., Qiaozhu M. Team Resilience under Shock: An Empirical Analysis of GitHub Repositoriesduring Early COVID-19 Pandemic // ICWSM, 2023. –V. 17. – 578-589 pp.
11. GHTorrent [Электронный ресурс]. URL: https://github.com/ghtorrent/ghtorrent.org/tree/master (дата обращения: 02.02.2024 г.).
12. Figma [Электронный ресурс]. URL: https://www.figma.com (дата обращения: 02.02.2024 г.).

ПРИЛОЖЕНИЕ

Интерфейсы функций

Листинг 1 – Интерфейс функции get_data

```
async def get_data(self, start_date: str,
                 end_date: str | None = None,
                 limit=1000,
                 repo_watch_event_count=10,
                 repo_members_count=3):
        """
        Асинхронный метод для получения данных из ClickHouse и GitHub API.

        Параметры:
        :param start_date: Начальная дата для запроса данных.
        :type start_date: str
        :param end_date: Конечная дата для запроса данных. Если не указана,
то используется start_date.
        :type end_date: str, optional
        :param limit: Ограничение на количество строк в запросе к ClickHouse.
        :type limit: int, optional
        :param repo_watch_event_count: Количество событий "watch" в
репозитории.
        :type repo_watch_event_count: int, optional
        :param repo_members_count: Количество участников в репозитории.
        :type repo_members_count: int, optional

        Возвращает:
        :return: DataFrame, содержащий данные из ClickHouse и GitHub API.
        :rtype: pandas.DataFrame
        """
```

Листинг 2 – Интерфейс функции get_quantiles

```
    def get_quantiles(self, start_date, end_date, table_columns, quantiles):
        """
        Метод для получения квантилей для указанных столбцов таблицы.

        Параметры:
        :param start_date: Начальная дата для запроса данных.
        :type start_date: str
        :param end_date: Конечная дата для запроса данных.
        :type end_date: str
```

```
        :param table_columns: Список столбцов таблицы, для которых нужно
получить квантили.
        :type table_columns: list
        :param quantiles: Список квантилей, которые нужно получить.
        :type quantiles: list

        Возвращает:
        :return: DataFrame, содержащий квантили для указанных столбцов.
        :rtype: pandas.DataFrame
        """
```

Листинг 3 – Интерфейс функции convert_data_to_transactions

```
def convert_data_to_transactions(self,
                                 repos_data:pd.DataFrame,
                                 quantiles_data: pd.DataFrame):
        """
        Метод для преобразования данных репозиториев в транзакции.

        Параметры:
        :param repos_data: Данные репозиториев.
        :type repos_data: pandas.DataFrame
        :param quantiles_data: Данные квантилей.
        :type quantiles_data: pandas.DataFrame

        Возвращает:
        :return: Транзакции.
        :rtype: pandas.DataFrame
        """
```

Листинг 4 – Интерфейс функции get_data_params

```
def mine_patterns(self,
                  data,
                  min_supp,
                  min_conf,
                  min_lift,
                  max_left_elements=3,
                  max_right_elements=1):
        """
        Метод для поиска паттернов в данных.
```

```
        Параметры:
        :param data: Данные для анализа.
        :type data: pandas.DataFrame
        :param min_supp: Минимальное значение поддержки.
        :type min_supp: float
        :param min_conf: Минимальное значение уверенности.
        :type min_conf: float
:param min_lift: Минимальное значение подъема.
        :type min_lift: float
        :param max_left_elements: Максимальное количество элементов в
антецеденте.
        :type max_left_elements: int
        :param max_right_elements: Максимальное количество элементов в
консеквенте.
        :type max_right_elements: int

        Возвращает:
        :return: Правила ассоциации.
        :rtype: pandas.DataFrame
        """
```